



Indexing

- query arrived, translate to relational algebra

15.3 → discussed implementation of selects

Selection

✅ A1 linear search → when we don't have index for city, city is not primary key

```
Select *  
From S  
where CITY = 'Paris' .  
and STATUS = 30
```

- we have to scan the entire table.
- the primary key of most implementation → like let said, primary key is s#, system will create a unique index to enforce the uniqueness property
 - so index serve 2 purpose
 - uniqueness property
 - speedup when access
- for non-key, you have create index statement → so you can have index on any column or on any combination of column.
- and in the case that your index and data or both sorted on the same key → clustered index.

if we don't have index → we have to scan the entire table = 'linear search'

- in the case that data is sorted without an index → you can use binary search → but very unlikely



Selection Operation

- **File scan**
- Algorithm **A1 (linear search)**. Scan each file block and test all records to see whether they satisfy the selection condition.
 - Cost estimate = b_r block transfers + 1 seek
 - b_r denotes number of blocks containing records from relation r
 - If selection is on a key attribute, can stop on finding record
 - cost = $(b_r/2)$ block transfers + 1 seek
 - Linear search can be applied regardless of
 - selection condition or
 - ordering of records in the file, or
 - availability of indices
- Note: binary search generally does not make sense since data is not stored consecutively
 - except when there is an index available,
 - and binary search requires more seeks than index search

cost estimate = b_r block transfers + 1 seek

- the estimation for non-key search because it could be anywhere → the entire table have to be search.

if selection is on key

- can stop searching when is found



normally, when we don't have index, the DBMS won't create temporary index just for the search, simply because to create an index, a full table scan is require → so better just scan

✓ A2 clustering(data is sorted) index, equality on key

- equality on key → we know that only 1 row return

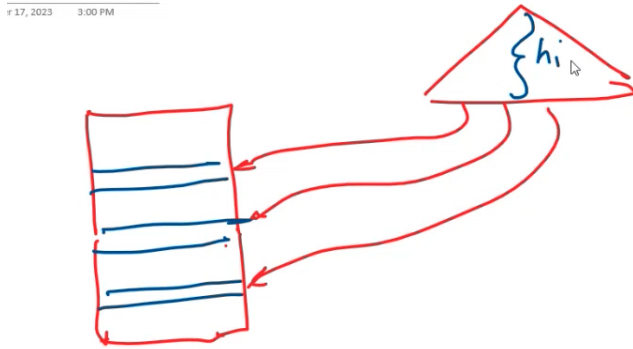


Selections Using Indices

- **Index scan** – search algorithms that use an index
 - selection condition must be on search-key of index.
- **A2 (clustering index, equality on key)**. Retrieve a single record that satisfies the corresponding equality condition
 - $Cost = (h_i + 1) * (t_T + t_S)$

- height → height of index tree

- when we organized index, we put it in tree format



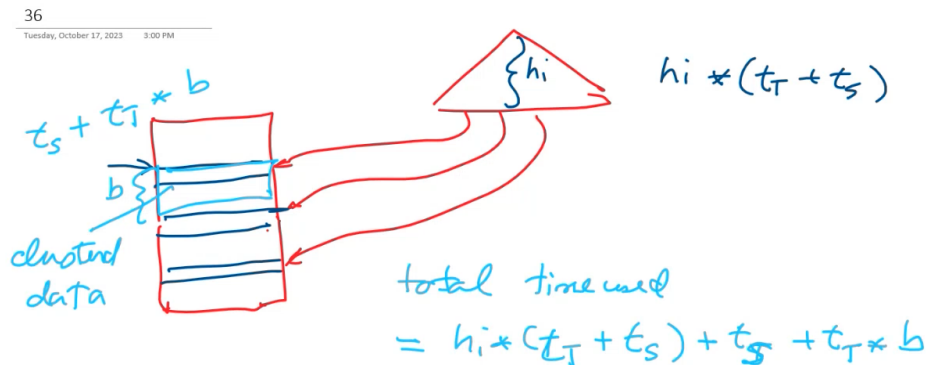
so for clustering index, equality on key

- the cost is height + 1, because it return only 1 row
- t_T is transfer time
- t_s is seek time to identify particular row

✓ A3 clustering index, equality on non-key

- **A3 (clustering index, equality on nonkey)** Retrieve multiple records.
 - Records will be on consecutive blocks
 - Let b = number of blocks containing matching records
 - $Cost = h_i * (t_T + t_s) + t_s + t_T * b$

- b is number of block containing matching record.



- cluster data → require 1 seek time + transfer time * block

- total time used $\rightarrow h_i * (t_T + t_s) + t_s(\text{one seek to the first record}) + t_T(\text{transfer time of the entire block}) * b$

👉 you need to search index first to find address $\rightarrow$ seek for address and read consecutive row (dbms)

How to find this height?

Go back to Basic Concept



Basic Concepts

- Indexing mechanisms used to speed up access to desired data.
 - E.g., author catalog in library
- **Search Key** - attribute to set of attributes used to look up records in a file.
- An **index file** consists of records (called **index entries**) of the form

search-key	pointer
------------	---------
- Index files are typically much smaller than the original file
- Two basic kinds of indices:
 - **Ordered indices:** search keys are stored in sorted order
 - **Hash indices:** search keys are distributed uniformly across "buckets" using a "hash function".

- we have data, we have separate index
- index point to data
- the term search key is an attribute or set of attribute use to look up record in a file
 - like if you look up for supplier in paris $\rightarrow$ city is a search key
- 2 basic kinds of indeices
 - order index
 - hash index
- now we talk about order index, which mean index must be sorted



Ordered Indices

- In an **ordered index**, index entries are stored sorted on the search key value.
- **Clustering index**: in a sequentially ordered file, the index whose search key specifies the sequential order of the file.
 - Also called **primary index**
 - The search key of a primary index is usually but not necessarily the primary key.

Dense Index



Dense Index Files

- **Dense index** — Index record appears for every search-key value in the file.
- E.g. index on *ID* attribute of *instructor* relation

10101	→	10101	Srinivasan	Comp. Sci.	65000	
12121	→	12121	Wu	Finance	90000	
15151	→	15151	Mozart	Music	40000	
22222	→	22222	Einstein	Physics	95000	
32343	→	32343	El Said	History	60000	
33456	→	33456	Gold	Physics	87000	
45565	→	45565	Katz	Comp. Sci.	75000	
58583	→	58583	Califieri	History	62000	
76543	→	76543	Singh	Finance	80000	
76766	→	76766	Crick	Biology	72000	
83821	→	83821	Brandt	Comp. Sci.	92000	
98345	→	98345	Kim	Elec. Eng.	80000	

- what is a dense index?
 - you have dense index on id number, all id number appear on dense index
- in fact, each index entry point to block address + some displacement within the block (database block is the smallest unit dbms will read from the disk)



Dense Index Files (Cont.)

- Dense index on *dept_name*, with *instructor* file sorted on *dept_name*

Biology	76766	Crick	Biology	72000
Comp. Sci.	10101	Srinivasan	Comp. Sci.	65000
Elec. Eng.	45565	Katz	Comp. Sci.	75000
Finance	83821	Brandt	Comp. Sci.	92000
History	98345	Kim	Elec. Eng.	80000
Music	12121	Wu	Finance	90000
Physics	76543	Singh	Finance	80000
	32343	El Said	History	60000
	58583	Califieri	History	62000
	15151	Mozart	Music	40000
	22222	Einstein	Physics	95000
	33465	Gold	Physics	87000

Sparse Index



Sparse Index Files

- Sparse Index:** contains index records for only some search-key values.
 - Applicable when records are sequentially ordered on search-key
- To locate a record with search-key value K we:
 - Find index record with largest search-key value $< K$
 - Search file sequentially starting at the record to which the index record points

10101	10101	Srinivasan	Comp. Sci.	65000
32343	12121	Wu	Finance	90000
76766	15151	Mozart	Music	40000
	22222	Einstein	Physics	95000
	32343	El Said	History	60000
	33456	Gold	Physics	87000
	45565	Katz	Comp. Sci.	75000
	58583	Califieri	History	62000
	76543	Singh	Finance	80000
	76766	Crick	Biology	72000
	83821	Brandt	Comp. Sci.	92000
	98345	Kim	Elec. Eng.	80000

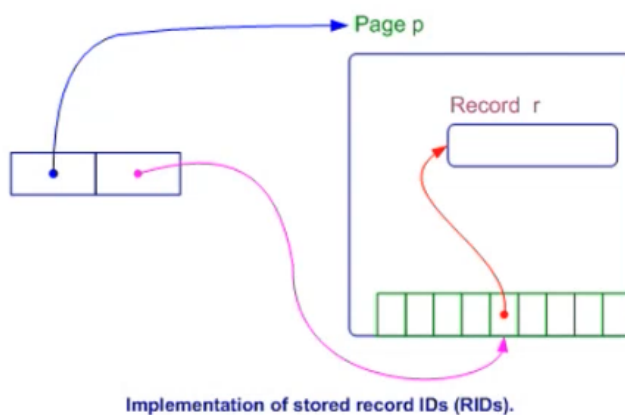
- don't have every index entry for every search key value.
- normally this pointer points to the beginning of database block
 - can of like point to database block and you further search that database block again.

Both index point to the beginning of the block → retrieve the block, find displacement inside

but in practice, dense index normally run faster on query on sparse index → even tho those indices point to the same place. (both point to the beginning of database block)

- why do query on dense index are normally faster than query on sparse index.

some more explanation



- when system read → it read the entire block not just the record in a block, so there should not be that much different, because we have to bring the entire block into the main memory anyway.

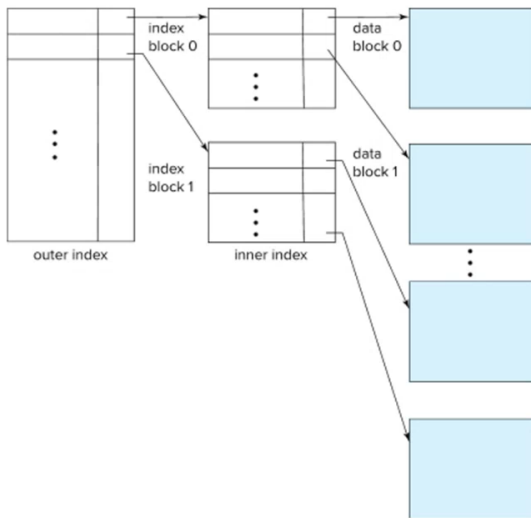


When you use dense index, you can use good search algorithm (binary search on index set) to find the record

- for sparse index, you only need scan, don't have all entry
- if you have millions, we choose organized index in tree format (not parallel like above)

✓ Multilevel index

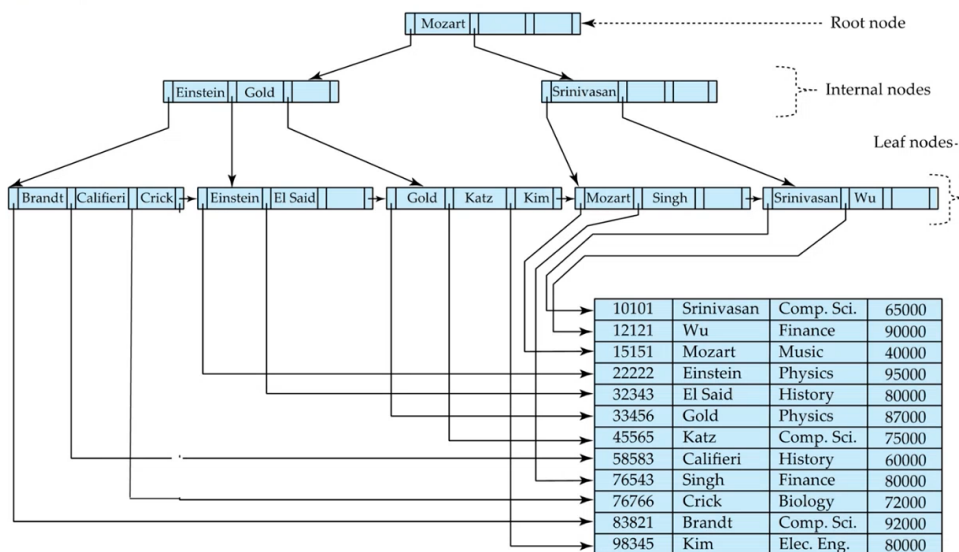
Multilevel Index (Cont.)



- in fact, this is a tree
- Multilevel index may not be a balance tree
- is it better to use balance tree?



Example of B⁺-Tree



$n = 4$



B⁺-Tree Node Structure

- Typical node

P_1	K_1	P_2	$\dots$	P_{n-1}	K_{n-1}	P_n
-------	-------	-------	---------	-----------	-----------	-------

- K_i are the search-key values
 - P_i are pointers to children (for non-leaf nodes) or pointers to records or buckets of records (for leaf nodes).
- The search-keys in a node are ordered

$$K_1 < K_2 < K_3 < \dots < K_{n-1}$$
 (Initially assume no duplicate keys, address duplicates later)

- you have pointer and key fit in one node (this is one db block, but some dbms allow many node?)



B⁺-Tree Index Files (Cont.)

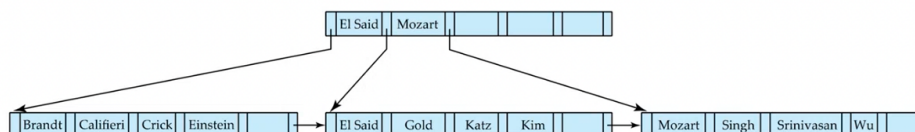
A B⁺-tree is a rooted tree satisfying the following properties:

- All paths from root to leaf are of the same length
- Each node that is not a root or a leaf has between $\lceil n/2 \rceil$ and n children.
- A leaf node has between $\lceil (n-1)/2 \rceil$ and $n-1$ values
- Special cases:
 - If the root is not a leaf, it has at least 2 children.
 - If the root is a leaf (that is, there are no other nodes in the tree), it can have between 0 and $(n-1)$ values.



Example of B⁺-tree

- B⁺-tree for *instructor* file ($n = 6$)



- Leaf nodes must have between 3 and 5 values ($\lceil (n-1)/2 \rceil$ and $n-1$, with $n = 6$).
- Non-leaf nodes other than root must have between 3 and 6 children ($\lceil n/2 \rceil$ and n with $n = 6$).
- Root must have at least 2 children.

- is the smaller the better, or larger the better

- $n = 2 \rightarrow$ binary tree
- $n = 1 \rightarrow$ linked list
- bigger n is better, less level to search

Formula to calculate the height of the tree



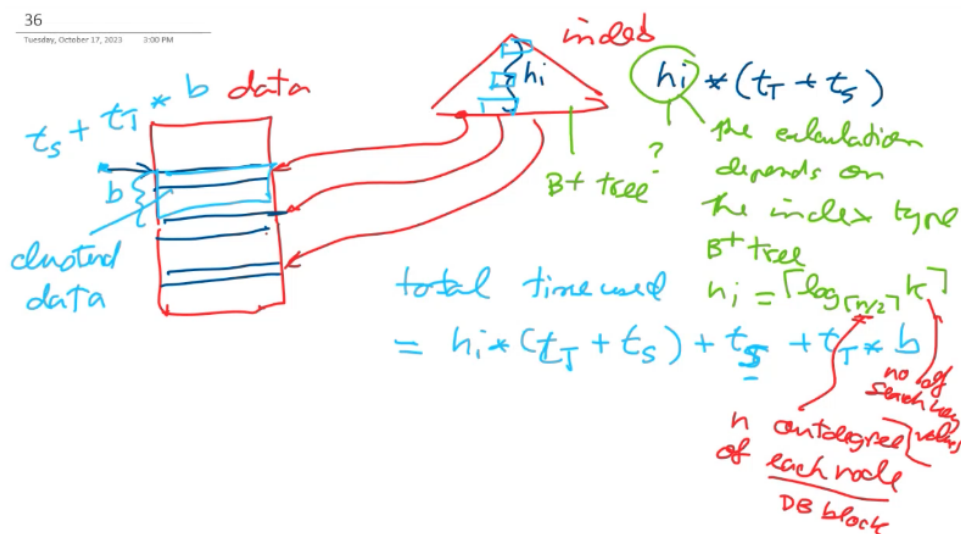
Observations about B⁺-trees

- Since the inter-node connections are done by pointers, “logically” close blocks need not be “physically” close.
- The non-leaf levels of the B⁺-tree form a hierarchy of sparse indices.
- The B⁺-tree contains a relatively small number of levels
 - Level below root has at least $2 \cdot \lceil n/2 \rceil$ values
 - Next level has at least $2 \cdot \lceil n/2 \rceil \cdot \lceil n/2 \rceil$ values
 - .. etc.
- If there are K search-key values in the file, the tree height is no more than $\lceil \log_{\lceil n/2 \rceil}(K) \rceil$
- thus searches can be conducted efficiently.

- how you obtain $n \rightarrow$ know block size, and size of search key
 - block size/search key $\rightarrow$ approximate number on x
- you have K search key $\rightarrow \log_{n/2}(K)$

36

Tuesday, October 17, 2023 3:00 PM



- transfer time and seek time will be given

- to calculate $n \rightarrow n$ is out degree $\rightarrow$ you need to know the size of database block and search key

✓ A4 secondary index, equality on key/non key (doesn't matter)

- **A4 (secondary index, equality on key/non-key).**
 - Retrieve a single record if the search-key is a candidate key
 - $Cost = (h_i + 1) * (t_T + t_S)$
 - Retrieve multiple records if search-key is not a candidate key
 - each of n matching records may be on a different block
 - $Cost = (h_i + n) * (t_T + t_S)$
 - Can be very expensive!

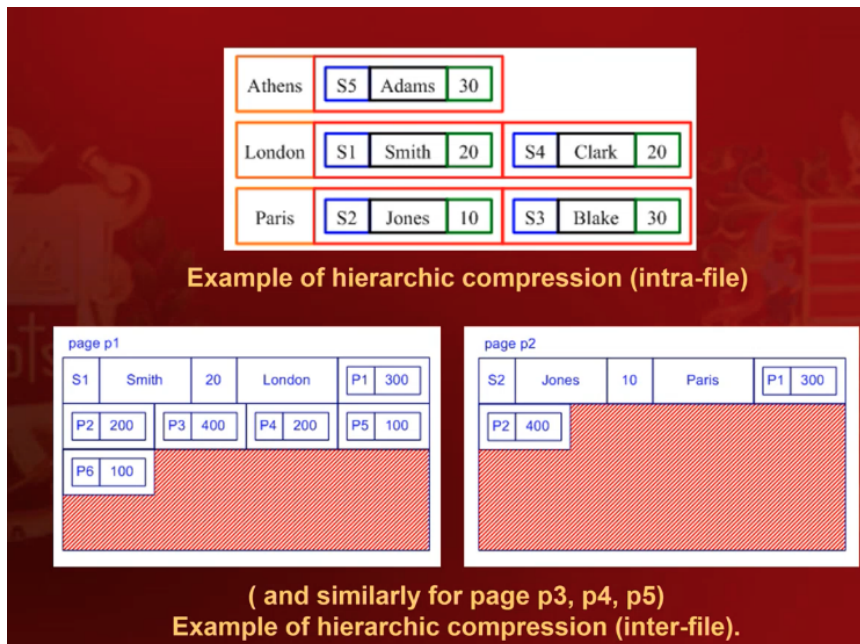
- data and index are not sorted in the same key.

not a candidate key

- more expensive than linear search.
- we assume the record we want can be anywhere
 - is secondary index rare? it anywhere (not rare) because primary index is cluster index, how many people cluster the table
 - way to improve $\rightarrow$ change from secondary index to primary index from the query you use often.

Cluster (physical database)

- to keep logically adjacent data(rows) together physically



upper one → show database block cluster by city, row that belong to same city group together

- single table cluster

lower one → not all dbms has it → multitable cluster (join?)

can a logical databaset able have more than 1 cluster physically?

- in practice, we only have 1 cluster.
- (other way) → replication have many copy (keep many copy of data)

To improve performance

- physically cluster your table based on frequence sql statement? (performance tuning)
- not just sqap sql statement

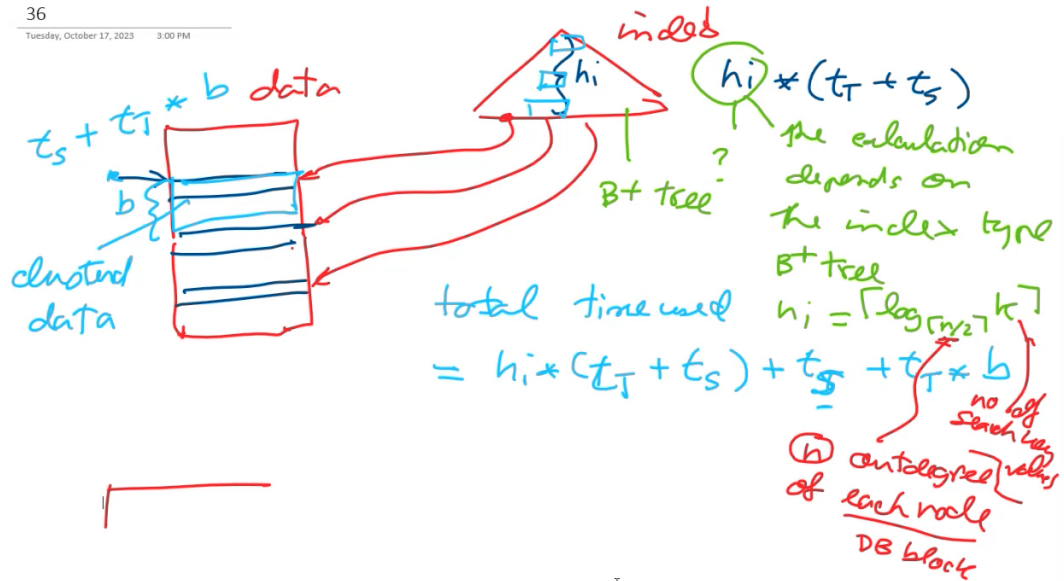
	Algorithm	Cost	Reason
A1	Linear Search	$t_S + b_r * t_T$	One initial seek plus b_r block transfers, where b_r denotes the number of blocks in the file.
A1	Linear Search, Equality on Key	Average case $t_S + (b_r/2) * t_T$	Since at most one record satisfies the condition, scan can be terminated as soon as the required record is found. In the worst case, b_r block transfers are still required.
A2	Clustering B ⁺ -tree Index, Equality on Key	$(h_i + 1) * (t_T + t_S)$	(Where h_i denotes the height of the index.) Index lookup traverses the height of the tree plus one I/O to fetch the record; each of these I/O operations requires a seek and a block transfer.
A3	Clustering B ⁺ -tree Index, Equality on Non-key	$h_i * (t_T + t_S) + t_S + b * t_T$	One seek for each level of the tree, one seek for the first block. Here b is the number of blocks containing records with the specified search key, all of which are read. These blocks are leaf blocks assumed to be stored sequentially (since it is a clustering index) and don't require additional seeks.
A4	Secondary B ⁺ -tree Index, Equality on Key	$(h_i + 1) * (t_T + t_S)$	This case is similar to clustering index.

- we do linear search when we don't have an index → scan entire table to look for some row that satisfy condition
 - it only need 1 seek to find the table and transfer time for all the block ($b_r * t_T$)
 - when we search non-key, more than 1 row may return (search from beginning to the end)
 - why we don't just create index and drop later
 - to create an index, full table scan need to be done → not worth it
- Linear Search equality on key
 - only 1 row return (once found → stop searching)
 - roughly estimate that to find a row → scan half of table
- A2 (clustering index that return only 1 row)
 - when you see the term clustering → the data is sorted
 - and row that have to same attribute value are group together.
 - city = london → you can assume there together

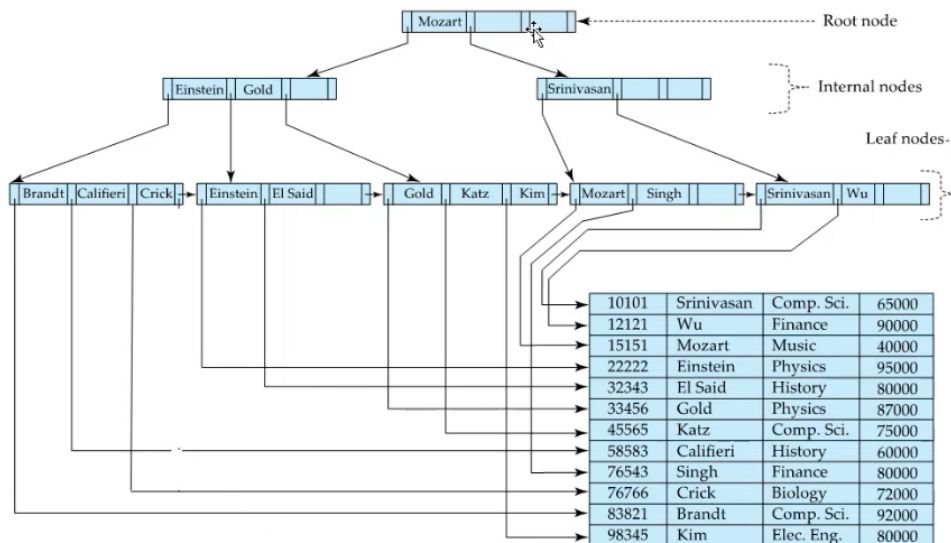
- because of that, block use to retrieve them → we can use number of row/blocking factor(number of row in a block?)
- cost is height * transfer time + seek time

36

Tuesday, October 17, 2023 3:00 PM



Example of B⁺-Tree



- $n = 4$
- k number of search key → lecturer name
- height = 3, for each node we need to find seek time and transfer time for each node?

- A3
 - $hi * (tT + ts) \rightarrow$ for index
 - for data, we know that the data are sorted together, so we seek the first one which satisfy the conditoon, the rest we can assume the row are together $\rightarrow$ so just use the block multiply by transfer time



Dense Index Files (Cont.)

- Dense index on *dept_name*, with *instructor* file sorted on *dept_name*

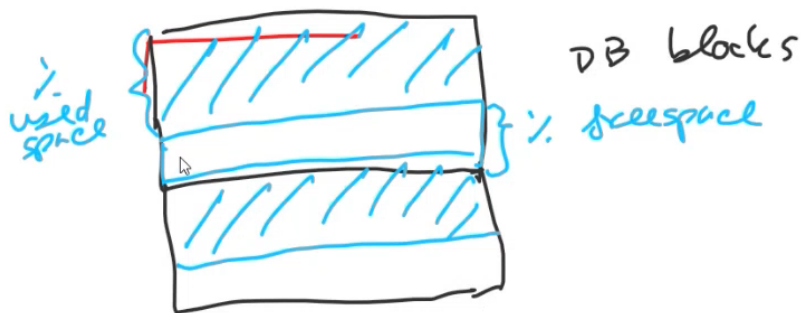
Biology	76766	Crick	Biology	72000
Comp.Sci.	10101	Srinivasan	Comp. Sci.	65000
Elec. Eng.	45565	Katz	Comp. Sci.	75000
Finance	83821	Brandt	Comp. Sci.	92000
History	98345	Kim	Elec. Eng.	80000
Music	12121	Wu	Finance	90000
Physics	76543	Singh	Finance	80000
	32343	El Said	History	60000
	58583	Califieri	History	62000
	15151	Mozart	Music	40000
	22222	Einstein	Physics	95000
	33465	Gold	Physics	87000

- find com sci $\rightarrow$ all comsci row are together
- A4 (equality on key) return only 1 row
 - A4, A2 are the same
- A4 (equality on non key)
 - but its n $\rightarrow$ this n is number of row return not the number of block
 - notice that we use block (cluster index) $\rightarrow$ we assume the row are together
 - we use n to multiply the transfer and seek time in this case.
 - cost is potentially very high if n is large
 - optimizer will compare this to A1 $\rightarrow$ many cases A1 win (scanning the entire table can be less costly than using this on non key) (if like more than 20% normally)

row has no order logically, but ohysically, it can be cluster

if you really want to improve, you may consider clustering your table by that column

- but the problem is
 - in database block you have percentage → free space, and percentage used
 - when we initially load the data, we leave free space for further insertion
 - if you don't have freespace (it full) → export import data, drop the table, create the table again, reorganize so it have space for insert (and still be cluster)
 - for cluster → you have to take care of them.



- **A5**
- **A6**

A5	Clustering B ⁺ -tree Index, Comparison	$h_i * (t_T + t_S) + t_S + b * t_T$	Identical to the case of A3, equality on non-key.
A6	Secondary B ⁺ -tree Index, Comparison	$(h_i + n) * (t_T + t_S)$	Identical to the case of A4, equality on non-key.

- comparison → not equality
 - find lecturer whose name between ____ and ____
- **A7 - conjunctive selection using one index**

15.3.3 Implementation of Complex Selections

So far, we have considered only simple selection conditions of the form $A \text{ op } B$, where op is an equality or comparison operation. We now consider more complex selection predicates.

- **Conjunction:** A **conjunctive selection** is a selection of the form:

$$\sigma_{\theta_1 \wedge \theta_2 \wedge \dots \wedge \theta_n}(r)$$

- **Disjunction:** A **disjunctive selection** is a selection of the form:

$$\sigma_{\theta_1 \vee \theta_2 \vee \dots \vee \theta_n}(r)$$

A disjunctive condition is satisfied by the union of all records satisfying the individual, simple conditions θ_i .

- **Negation:** The result of a selection $\sigma_{\neg\theta}(r)$ is the set of tuples of r for which the condition θ evaluates to false. In the absence of nulls, this set is simply the set of tuples in r that are not in $\sigma_{\theta}(r)$.

We can implement a selection operation involving either a conjunction or a disjunction of simple conditions by using one of the following algorithms:

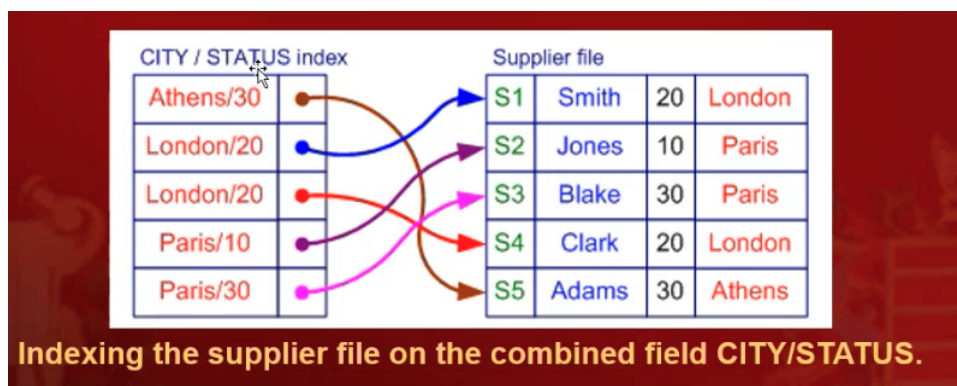
- **A7 (conjunctive selection using one index).** We first determine whether an access path is available for an attribute in one of the simple conditions. If one is, one of the

- even tho we may have more than 1 index
- a 7 use only 1 idex
- ex: select * from supplier where city = paris and status = 30
 - you can choose 1 of them (you may choose city first)

• A8

- **A8 (conjunctive selection using composite index).** An appropriate **composite index** (that is, an index on multiple attributes) may be available for some conjunctive selections. If the selection specifies an equality condition on two or more attributes, and a composite index exists on these combined attribute fields, then the index can be searched directly. The type of index determines which of algorithms A2, A3, or A4 will be used.

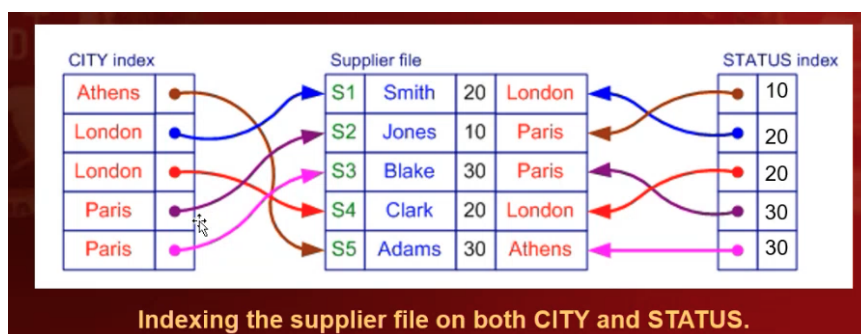
ex of composite index → it will sort value according to the index (sort by city first then status)



- **A9**

- **A9 (conjunctive selection by intersection of identifiers).** Another alternative for implementing conjunctive selection operations involves the use of record pointers or record identifiers. This algorithm requires indices with record pointers, on the fields involved in the individual conditions. The algorithm scans each index for pointers to tuples that satisfy an individual condition. The intersection of all the retrieved pointers is the set of pointers to tuples that satisfy the conjunctive condition. The algorithm then uses the pointers to retrieve the actual records. If indices are not available on all the individual conditions, then the algorithm tests the retrieved records against the remaining conditions.

The cost of algorithm A9 is the sum of the costs of the individual index scans, plus the cost of retrieving the records in the intersection of the retrieved lists of pointers. This cost can be reduced by sorting the list of pointers and retrieving records in the sorted order. Thereby, (1) all pointers to records in a block come



- find intersection between 2 index (identifier?)
- intersection of identifier give us a common one

- **A10**

- **A10 (disjunctive selection by union of identifiers).** If access paths are available on all the conditions of a disjunctive selection, each index is scanned for pointers to tuples that satisfy the individual condition. The union of all the retrieved pointers yields the set of pointers to all tuples that satisfy the disjunctive condition. We then use the pointers to retrieve the actual records.

However, if even one of the conditions does not have an access path, we have to perform a linear scan of the relation to find tuples that satisfy the condition.

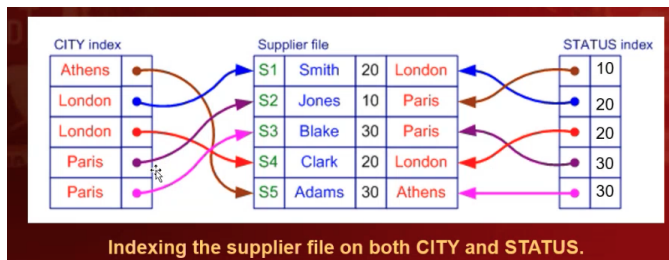
- disjunction → or
- if query is an "or"

Select *

From S

where CITY = 'Paris'

or status = 30



- we find the pointer (identifier) that point to paris, the pointer that point to 30
- and we union them → get all

if one condition doesnot have access path (no index) → we have to perform a linear scan of the relation to find tuples that satisfy the condition. Therefore, if there is even one such condition in the disjunct, the most efficient access method is a linear scan.....

- people complain about condition wiht or → that its slow → yes if some condition that have an index.

Join

15.5 Join Operation

In this section, we study several algorithms for computing the join of relations, and we analyze their respective costs.

We use the term **equi-join** to refer to a join of the form $r \bowtie_{r.A=s.B} s$, where A and B are attributes or sets of attributes of relations r and s , respectively.

We use as a running example the expression:

$student \bowtie_{s_{id}} takes$

using the same relation schemas that we used in Chapter 2. We assume the following information about the two relations:

- Number of records of *student*: $n_{student} = 5000$.
- Number of blocks of *student*: $b_{student} = 100$.
- Number of records of *takes*: $n_{takes} = 10,000$.
- Number of blocks of *takes*: $b_{takes} = 400$.

first algo

1. nested loop join

```
for each tuple  $t_r$  in  $r$  do begin
  for each tuple  $t_s$  in  $s$  do begin
    test pair  $(t_r, t_s)$  to see if they satisfy the join condition  $\theta$ 
    if they do, add  $t_r \cdot t_s$  to the result;
  end
end
```

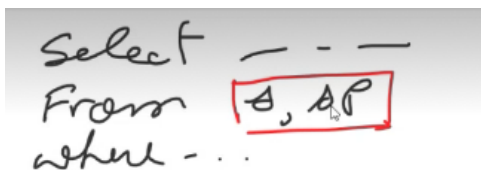
Figure 15.5 Nested-loop join.

r - small letter $\rightarrow$ instances

- capital letter $\rightarrow$ schema

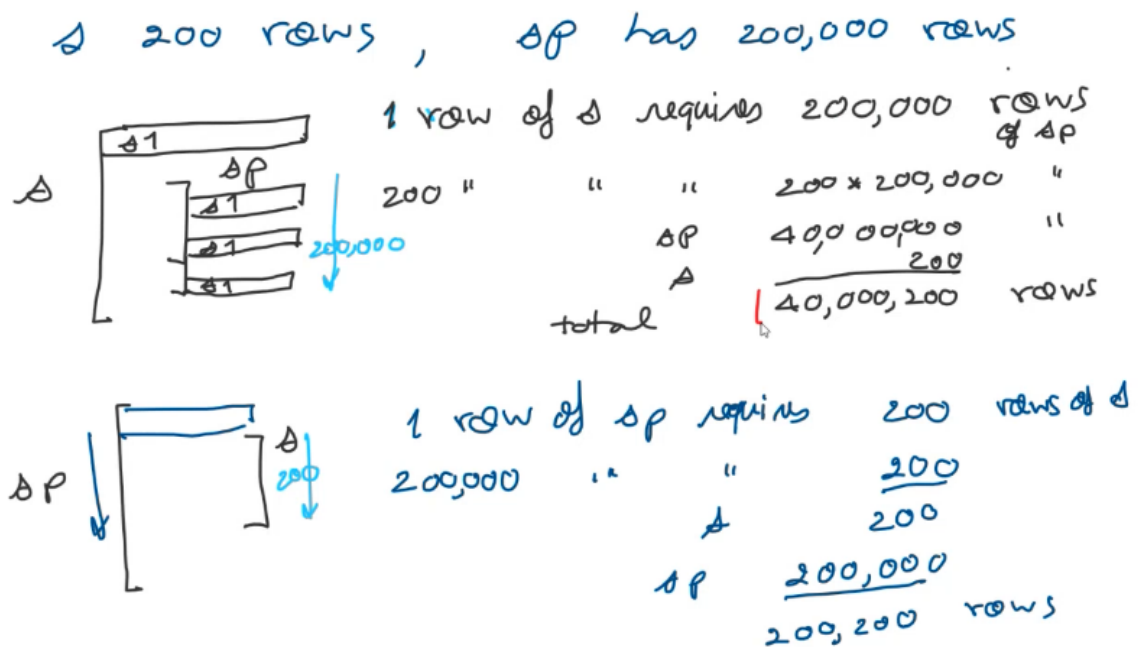
which one should be write first (s or sp)

- now we use cost based



Handwritten SQL query:
Select ---
From S, AP
where ---

- which one should be on outer loop, or inner loop
 - is it make different? yes



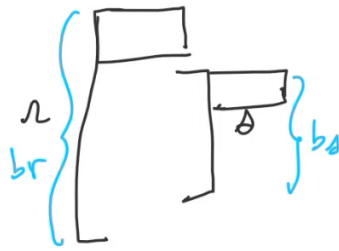
- the one in inner loop should be small enough to be in main memory during processing
- in the past, inner outer loop depend on from statement (first $\rightarrow$ maybe inner loop (not a fix rule, depend on rule base optimizer of particular product))
- in the past, main memory is very expensive
- this algo to like use least main memory??

```

for each block  $B_r$  of  $r$  do begin
  for each block  $B_s$  of  $s$  do begin
    for each tuple  $t_r$  in  $B_r$  do begin
      for each tuple  $t_s$  in  $B_s$  do begin
        test pair  $(t_r, t_s)$  to see if they satisfy the join condition
        if they do, add  $t_r \cdot t_s$  to the result;
      end
    end
  end
end
end
end

```

Figure 15.6 Block nested-loop join.



1 block of r requires b_s blocks of s
 b_r " " $b_r * b_s$ " —

$$total = b_r + (b_r * b_s) \text{ blocks}$$

- which one should be on inner loop → smaller or bigger
 - smaller outside is better

$$total = b_r + \underline{(b_r * b_s)} \text{ blocks}$$

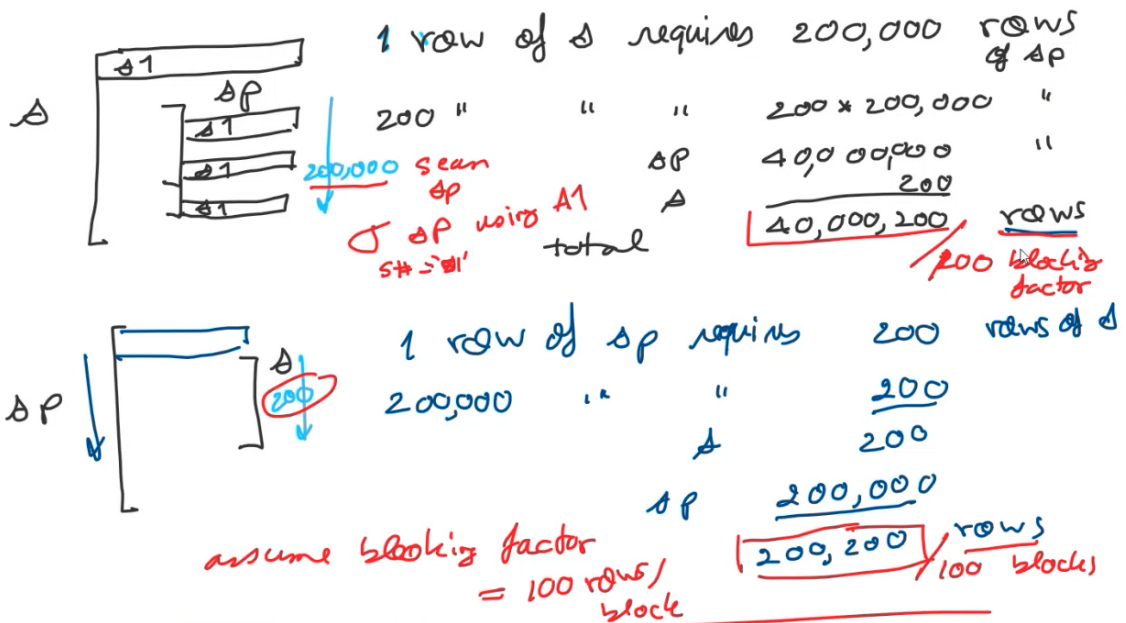
$$total = b_s + \underline{(b_s * b_r)} \text{ blocks}$$

if any table is small enough to be in a main memory →
 system use nested loop join, if not → without index, will
 use block nested-loop join

nowadays, for cost-based → fully automatic

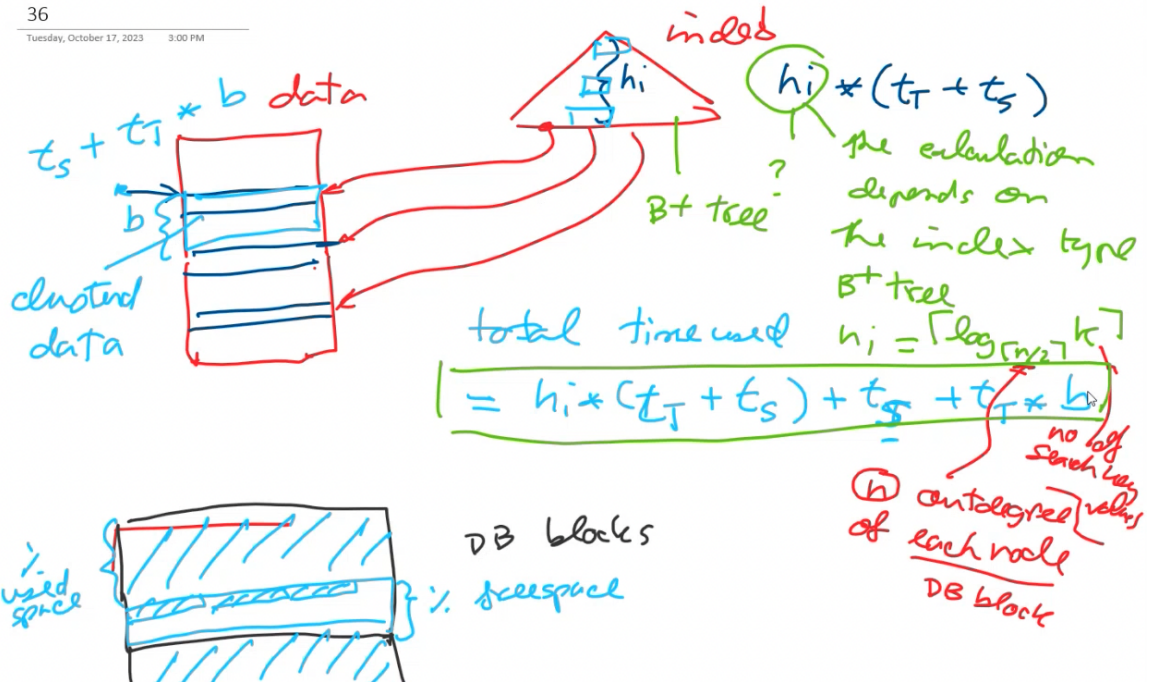
2. Indexed Nested loop join (use most nowadays)

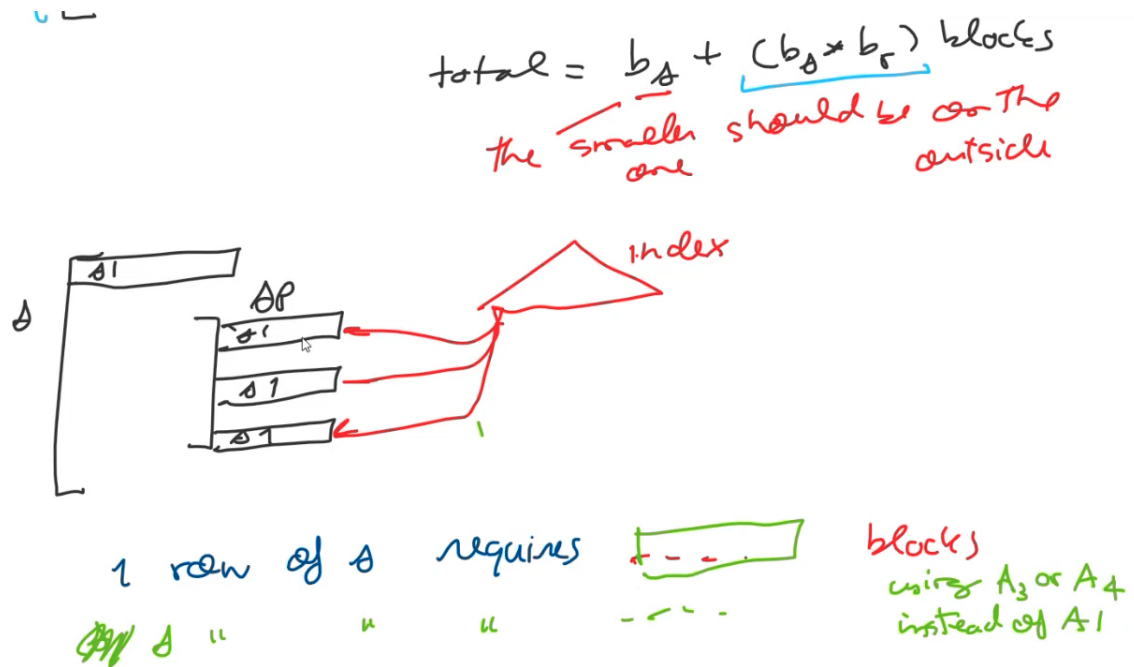
- in above we have to scan the whole sp table because no index (A1 linear search)



36

Tuesday, October 17, 2023 3:00 PM





- use index to calculate the number of block require for outer loop (swap → calculate)
 - choose (less block)
- if only has statistics, system might collect other on the fly → slow down

Summary

- you have input statement in a non-procedure format (SQL) → more like declarative

```

3) Select SNAME
   From S
   where CITY = 'London'
   and exists (
     Select *
     From SP
     where SP.S# = S.S#
     and P# = P21 \

```

- you may express sql in subquery format, correlate subquery?
- convert to relational algebra statements
- and like there will be many way (many plan)
- system has to determine which to be used based on physical condition (available of index, type of index,...)
- at the end, it give you the best result.